

# КОЛЛЕКЦИИ В KOTLIN

## Три типа коллекций

**List** — упорядоченная коллекция с доступом по индексу. Элементы могут повторяться.

**Set** — коллекция уникальных элементов. Дубликаты автоматически игнорируются.

**Map** — коллекция пар ключ-значение. Ключи уникальны.

```
val list = listOf(1, 2, 2, 3)      // [1, 2, 2, 3] – дубликаты сохраняются
val set = setOf(1, 2, 2, 3)       // [1, 2, 3] – дубликат убран
val map = mapOf("a" to 1, "b" to 2) // {a=1, b=2}
```

---

## Зачем столько видов коллекций?

### Read-only vs Mutable — зачем?

**Проблема:** если все коллекции изменяемые, легко сломать данные случайно:

```
// Представь, что любой код может изменить твой список
fun processUsers(users: MutableList<User>) {
    users.clear() // Упс, кто-то удалил всех пользователей
}
```

**Решение:** по умолчанию коллекции read-only — их нельзя изменить:

```
fun processUsers(users: List<User>) {
    // users.clear() // ОШИБКА КОМПИЛЯЦИИ – нет такого метода
}
```

**Правило:** используй List, Set, Map по умолчанию. Используй MutableList, MutableSet, MutableMap только когда реально нужно изменять.

---

## ArrayList vs LinkedList — зачем?

| Операция                  | ArrayList | LinkedList |
|---------------------------|-----------|------------|
| Доступ по индексу list[i] | Быстро    | Медленно   |
| Добавление в конец        | Быстро    | Быстро     |
| Вставка в середину        | Медленно  | Быстро     |
| Удаление из середины      | Медленно  | Быстро     |

**Когда что:** - **ArrayList (по умолчанию)** — почти всегда. Доступ по индексу  $O(1)$  - **LinkedList** — редко. Только если постоянно вставляешь/удаляешь в середине

```
val default = mutableListOf(1, 2, 3) // ArrayList
val linked = LinkedList(listOf(1, 2, 3)) // LinkedList – редко нужен
```

## HashSet vs LinkedHashSet vs TreeSet — зачем?

| Тип           | Порядок            | Скорость      | Когда использовать                    |
|---------------|--------------------|---------------|---------------------------------------|
| HashSet       | Не гарантирован    | Самый быстрый | Важна только уникальность             |
| LinkedHashSet | Порядок добавления | Быстрый       | Нужен порядок вставки (по умолчанию!) |
| TreeSet       | Отсортированный    | Медленнее     | Нужна автосортировка                  |

```
val hash = hashSetOf(3, 1, 2) // порядок может быть любой
val linked = linkedSetOf(3, 1, 2) // [3, 1, 2] – порядок добавления
val tree = sortedSetOf(3, 1, 2) // [1, 2, 3] – отсортировано

val default = setOf(3, 1, 2) // LinkedHashSet! Порядок сохранится
```

## HashMap vs LinkedHashMap vs TreeMap — зачем?

Та же логика:

| Тип           | Порядок ключей     | Когда использовать             |
|---------------|--------------------|--------------------------------|
| HashMap       | Не гарантирован    | Важна только скорость          |
| LinkedHashMap | Порядок добавления | Нужен порядок (по умолчанию!)  |
| TreeMap       | Отсортированный    | Нужна автосортировка по ключам |

```
val default = mapOf("c" to 1, "a" to 2) // LinkedHashMap – порядок сохранится
val sorted = sortedMapOf("c" to 1, "a" to 2) // {a=2, c=1} – по ключам
```

## Краткая шпаргалка: что выбрать?

Нужна коллекция?

- Нужен доступ по индексу? → List
  - └─ listOf() / mutableListOf()
- Нужна уникальность? → Set
  - └─ setOf() / mutableSetOf()
- Нужны пары ключ-значение? → Map
  - └─ mapOf() / mutableMapOf()

Нужно изменять?

- └ Нет → listOf, setOf, mapOf (read-only)
- └ Да → mutableListOf, mutableSetOf, mutableMapOf

---

## Read-only vs Mutable

### Read-only коллекции

```
val list = listOf(1, 2, 3)
val set = setOf(1, 2, 3)
val map = mapOf("a" to 1)
```

```
// list.add(4) // ОШИБКА – нет такого метода
```

### Важно: read-only ≠ immutable!

Read-only — это интерфейс без методов изменения. Но данные могут измениться через mutable ссылку:

```
val mutable = mutableListOf(1, 2, 3)
val readOnly: List<Int> = mutable // read-only VIEW
```

```
mutable.add(4)
println(readOnly) // [1, 2, 3, 4] – изменилось!
```

Если нужна настоящая копия: mutable.toList()

---

### Mutable коллекции

```
val list = mutableListOf(1, 2, 3)
list.add(4)
list.remove(2)
list[0] = 10
```

```
val set = mutableSetOf(1, 2, 3)
set.add(4)
```

```
val map = mutableMapOf("a" to 1)
map["b"] = 2
map.remove("a")
```

---

val защищает ссылку, не содержимое

```
val list = mutableListOf(1, 2, 3)
list.add(4) // OK — меняем содержимое
// list = mutableListOf(5, 6) // ОШИБКА — нельзя переназначить
```

---

## Создание коллекций

### Стандартные функции

```
// Read-only
val list = listOf(1, 2, 3)
val set = setOf(1, 2, 3)
val map = mapOf("a" to 1, "b" to 2)

// Mutable
val mutableList = mutableListOf(1, 2, 3)
val mutableSet = mutableSetOf(1, 2, 3)
val mutableMap = mutableMapOf("a" to 1)

// Пустые
val empty = emptyList<Int>()
val emptySet = emptySet<String>()
val emptyMap = emptyMap<String, Int>()
```

### Builder функции

Когда нужно построить коллекцию с логикой:

```
val list = buildList {
    add(1)
    if (condition) add(2)
    addAll(otherList)
}

val map = buildMap {
    put("a", 1)
    put("b", 2)
}
```

---

## Доступ к элементам

### List — по индексу

```
val list = listOf("a", "b", "c")

list[0] // "a"
```

```
list.first()    // "a"  
list.last()     // "c"  
list.size      // 3
```

## Set — проверка наличия

```
val set = setOf(1, 2, 3)
```

```
1 in set        // true  
set.contains(1) // true  
set.size        // 3
```

## Map — по ключу

```
val map = mapOf("a" to 1, "b" to 2)
```

```
map["a"]        // 1  
map["z"]        // null  
"a" in map      // true (проверка ключа)  
map.keys        // [a, b]  
map.values      // [1, 2]
```

---

## Безопасный доступ к элементам

Функции с **OrNull** возвращают null вместо исключения:

```
val list = listOf(1, 2, 3)  
val empty = emptyList<Int>()
```

```
// Опасно  
empty.first() // NoSuchElementException!
```

```
// Безопасно  
empty.firstOrNull() // null  
list.firstOrNull { it > 10 } // null
```

```
// По индексу  
list.getOrNull(10) // null  
list.getOrElse(10) { 0 } // 0 (дефолт)
```

---

## Операции над коллекциями

Все операции возвращают **новую** коллекцию, не изменяя исходную.

### filter — отфильтровать

```
val numbers = listOf(1, 2, 3, 4, 5)
numbers.filter { it % 2 == 0 } // [2, 4]
numbers.filterNot { it % 2 == 0 } // [1, 3, 5]
```

### map — преобразовать

```
numbers.map { it * 2 } // [2, 4, 6, 8, 10]
numbers.map { it.toString() } // ["1", "2", "3", "4", "5"]
```

### sorted — отсортировать

```
val nums = listOf(3, 1, 4, 1, 5)
nums.sorted() // [1, 1, 3, 4, 5]
nums.sortedDescending() // [5, 4, 3, 1, 1]
```

```
data class User(val name: String, val age: Int)
val users = listOf(User("Bob", 25), User("Alice", 30))
users.sortedBy { it.name } // [Alice, Bob]
users.sortedByDescending { it.age } // [Alice(30), Bob(25)]
```

### take / drop — взять / пропустить

```
val list = listOf(1, 2, 3, 4, 5)
```

```
list.take(3) // [1, 2, 3]
list.drop(2) // [3, 4, 5]
list.takeLast(2) // [4, 5]
list.dropLast(2) // [1, 2, 3]
```

```
list.takeWhile { it < 3 } // [1, 2]
list.dropWhile { it < 3 } // [3, 4, 5]
```

### distinct — убрать дубликаты

```
listOf(1, 2, 2, 3, 3).distinct() // [1, 2, 3]
```

### flatten / flatMap — развернуть вложенные

```
val nested = listOf(listOf(1, 2), listOf(3, 4))
nested.flatten() // [1, 2, 3, 4]
```

```
nested.flatMap { it.map { n -> n * 2 } } // [2, 4, 6, 8]
```

### groupBy — сгруппировать

```
val words = listOf("apple", "banana", "apricot", "cherry")
words.groupBy { it.first() }
// {a=[apple, apricot], b=[banana], c=[cherry]}
```

any / all / none — проверки

```
val numbers = listOf(1, 2, 3, 4, 5)
```

```
numbers.any { it > 3 } // true — есть хотя бы один > 3
```

```
numbers.all { it > 0 } // true — все > 0
```

```
numbers.none { it < 0 } // true — нет отрицательных
```

fold / reduce — свернуть в одно значение

```
val numbers = listOf(1, 2, 3, 4)
```

```
numbers.sum() // 10
```

```
numbers.average() // 2.5
```

```
// fold — с начальным значением
```

```
numbers.fold(0) { acc, n -> acc + n } // 10
```

```
numbers.fold(1) { acc, n -> acc * n } // 24
```

```
// reduce — без начального (первый элемент = начальный)
```

```
numbers.reduce { acc, n -> acc + n } // 10
```

---

## Операторы + и - для коллекций

```
val list = listOf(1, 2, 3)
```

```
list + 4 // [1, 2, 3, 4]
```

```
list + listOf(4, 5) // [1, 2, 3, 4, 5]
```

```
list - 2 // [1, 3]
```

```
list - listOf(1, 2) // [3]
```

Для Map:

```
val map = mapOf("a" to 1, "b" to 2)
```

```
map + ("c" to 3) // {a=1, b=2, c=3}
```

```
map - "a" // {b=2}
```

---

## Операции с индексами

Когда нужен индекс — используйте функции с **Indexed**:

```
val list = listOf("a", "b", "c")
```

```
list.forEachIndexed { index, value ->  
    println("$index: $value")  
}
```

```
list.mapIndexed { index, value -> "$index-$value" }  
// ["0-a", "1-b", "2-c"]  
  
list.filterIndexed { index, _ -> index % 2 == 0 }  
// ["a", "c"]
```

---

## Преобразование List в Map

**associateBy** — ключ из элемента

```
data class User(val id: Int, val name: String)  
val users = listOf(User(1, "Alice"), User(2, "Bob"))
```

```
users.associateBy { it.id }  
// {1=User(1, Alice), 2=User(2, Bob)}
```

```
users.associateBy({ it.id }, { it.name })  
// {1="Alice", 2="Bob"}
```

**associateWith** — значение из элемента

```
val names = listOf("Alice", "Bob")  
names.associateWith { it.length }  
// {"Alice"]=5, "Bob"]=3}
```

**associate** — полный контроль

```
users.associate { it.name to it.id }  
// {"Alice"]=1, "Bob"]=2}
```

---

## Копирование коллекций

`toList()`, `toSet()`, `toMutableList()` создают **shallow copy**:

```
val original = listOf(User("Alice"))  
val copy = original.toList()
```

```
// Это разные списки  
original === copy // false
```

```
// Но объекты внутри — те же самые!  
original[0] === copy[0] // true
```

Для глубокой копии — копируй элементы вручную:

```
val deepCopy = original.map { it.copy() }
```



---

## Nullable коллекции

```
val list1: List<String>? = null      // коллекция может быть null
val list2: List<String?> = listOf("a", null) // элементы могут быть null
val list3: List<String?>? = null    // и то, и другое
```

Работа с ними:

```
val items: List<String>? = getItems()
items?.forEach { println(it) }

val items2: List<String?> = listOf("a", null)
items2.filterNotNull() // ["a"]
```

---

## Sequence — ленивые вычисления

Для больших данных или длинных цепочек операций:

```
// List — создаёт промежуточные списки
val result1 = list
    .filter { it > 2 } // создаёт список
    .map { it * 2 }    // создаёт ещё список

// Sequence — вычисляет лениво
val result2 = list.asSequence()
    .filter { it > 2 }
    .map { it * 2 }
    .toList() // вычисление здесь
```

**Когда использовать:** большие коллекции (тысячи элементов) или много операций подряд.

---

## Java Interop

Kotlin коллекции основаны на Java, но есть нюансы:

```
val kotlinList = listOf(1, 2, 3)
javaMethod(kotlinList) // передаём в Java

// Если Java попытается изменить:
// kotlinList.add(4) → UnsupportedOperationException
```

**Защита от Java:**

```
val javaList = javaMethod() // вернул List из Java
val safe = javaList.toList() // копия – теперь безопасно
```

**Platform types:**

```
val fromJava = javaMethod() // mun List<String!> – nullable неизвестен
val safe: List<String> = fromJava.filterNotNull()
```